TEAM 9: REFACTORING

COURSE: SOEN 6441 (WINTER 21)

INSTRUCTOR- JOEY PAQUET

TEAM MEMBERS:

1. Jayasurya Pazhani
2. Jayanth Apagundi
3. Shuvanidhi Suresh
4. Shahul Hameed P T S
5. Vinisha Kora Venkatesalu

Potential Refactoring Targets:

The following list of refactoring targets have been taken mainly from the new requirements established in build 2, and based on pain points and inconsistencies encountered during the development of build 1.

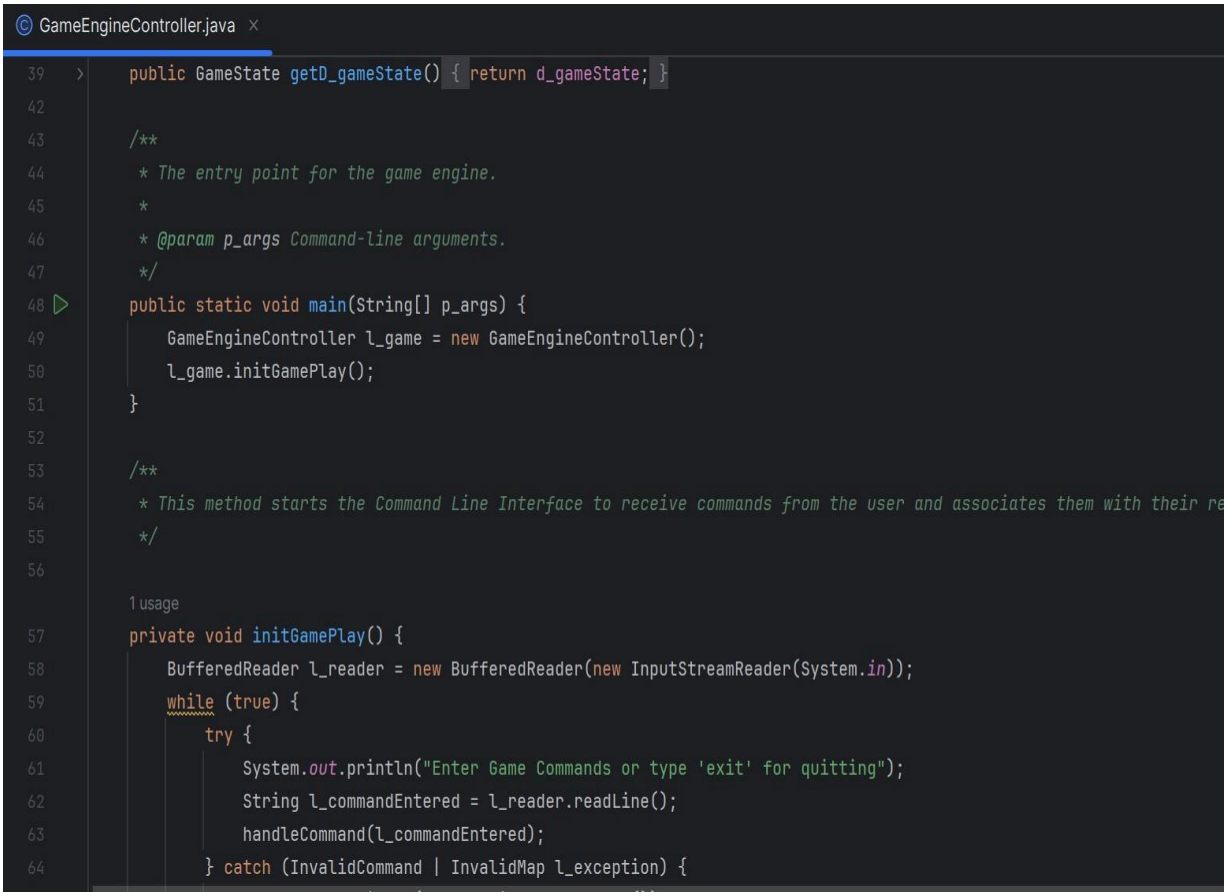
1. Implement the phases of the application, including the phases using State pattern.
2. Implementation of a game log file using the Observer pattern.
3. Use Command pattern to implement the Orders.
4. Identify and decouple dependencies.
5. Enhancing Exception Handling.
6. Improve Code Coverage with Additional Tests.
7. Encapsulate data members with appropriate access modifiers to enforce data integrity and information hiding.
8. Decompose large methods or classes into smaller, cohesive components to improve code organization.
9. Ensure consistent formatting and use meaningful names to enhance code readability Change Continent Check in Map validation.
10. Optimizing loops for better performance.
11. Improve User Interaction.
12. Organize code into logical sections, and group related methods together
13. Remove Redundant code blocks.
14. Check for unused variables, methods or classes.
15. Simplify complex conditional logic.

Actual Refactoring Targets:

The list of actual refactoring taken from the target list above were chosen mainly because of the new requirements established in build 2, and on the greatest pain points and inconsistencies encountered during the development of build 1.

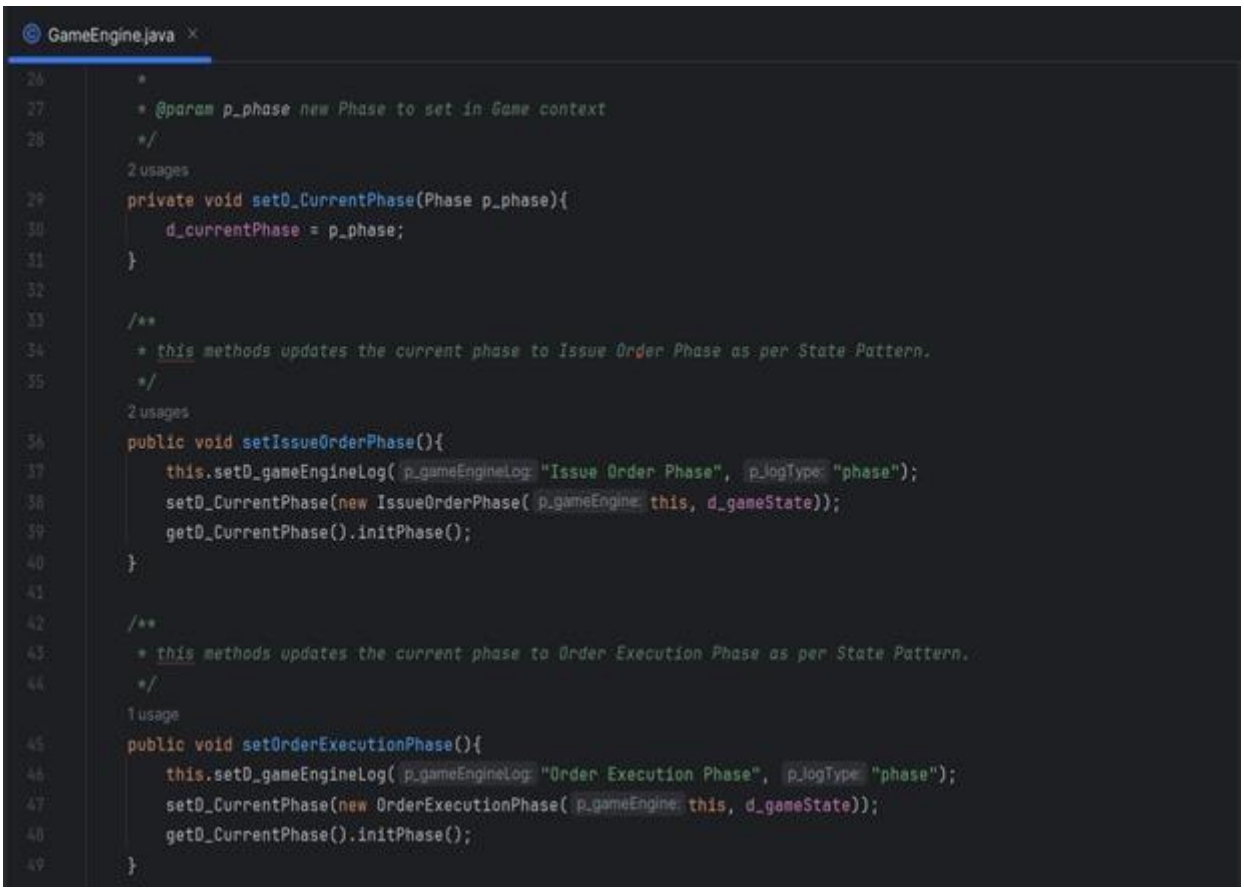
1.Implement the phases of the application, including the phases using State pattern

Before: The code had a monolithic structure where different phases of the game (e.g., startup, issue order, order execution) were handled within the same methods or classes.



Before

After: With the State pattern, the GameEngine class should now have a Phase field representing the current phase of the game. Each phase is encapsulated within its own class (StartupPhase, IssueOrderPhase, OrderExecutionPhase, etc.), implementing a common Phase interface or abstract class. The GameEngine class delegates phase-specific behavior to the current phase object.



2. Implementation of a game log file using the Observer pattern:

Before: In the original version, when an unrecognized action is encountered, a message is printed to the console using System.out.println.

Before

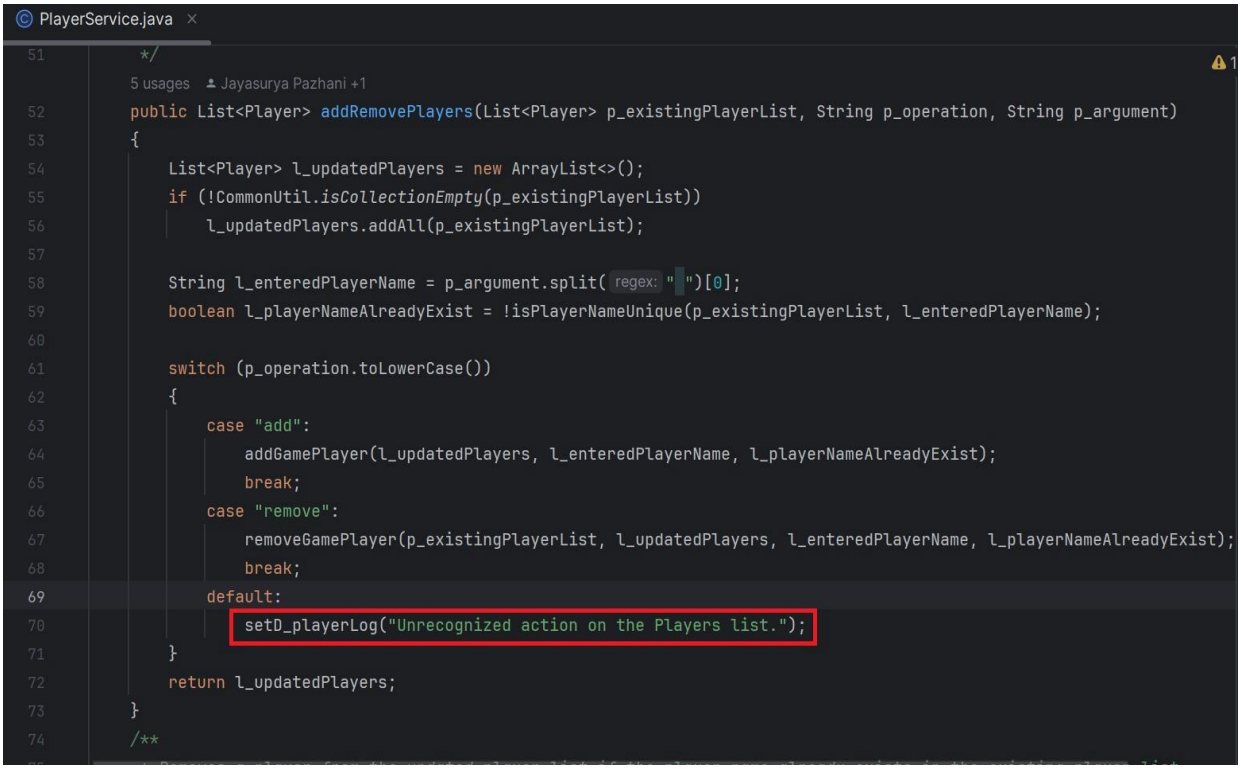
PlayerService.java

```
5 usages
public List<Player> addRemovePlayers(List<Player> p_existingPlayerList, String p_operation, String p_argument)
{
    List<Player> l_updatedPlayers = new ArrayList<>();
    if (!CommonUtil.isCollectionEmpty(p_existingPlayerList))
        l_updatedPlayers.addAll(p_existingPlayerList);

    String l_enteredPlayerName = p_argument.split( regex: " ")[0];
    boolean l_playerNameAlreadyExist = !isPlayerNameUnique(p_existingPlayerList, l_enteredPlayerName);

    switch (p_operation.toLowerCase())
    {
        case "add":
            addGamePlayer(l_updatedPlayers, l_enteredPlayerName, l_playerNameAlreadyExist);
            break;
        case "remove":
            removeGamePlayer(p_existingPlayerList, l_updatedPlayers, l_enteredPlayerName, l_playerNameAlreadyExist);
            break;
        default:
            System.out.println("Unrecognized action on the Players list.");
    }
    return l_updatedPlayers;
}
/**
 * Removes a player from the updated player list if the player name already exists in the existing player list.
```

After: In the refactored version, instead of directly printing the message to the console, a method setD_playerLog is called to set a log message. This change likely indicates a broader architectural decision to manage and handle log messages more systematically within the codebase.



After

3. Use Command pattern to implement the Orders:

Before: In the "before" version, the code directly handles orders within the Player class. The issueOrder method takes an order type and details as parameters and executes the corresponding logic based on the order type. This approach can lead to tight coupling between order execution logic and the Player class, making it harder to extend or modify the code in the future.

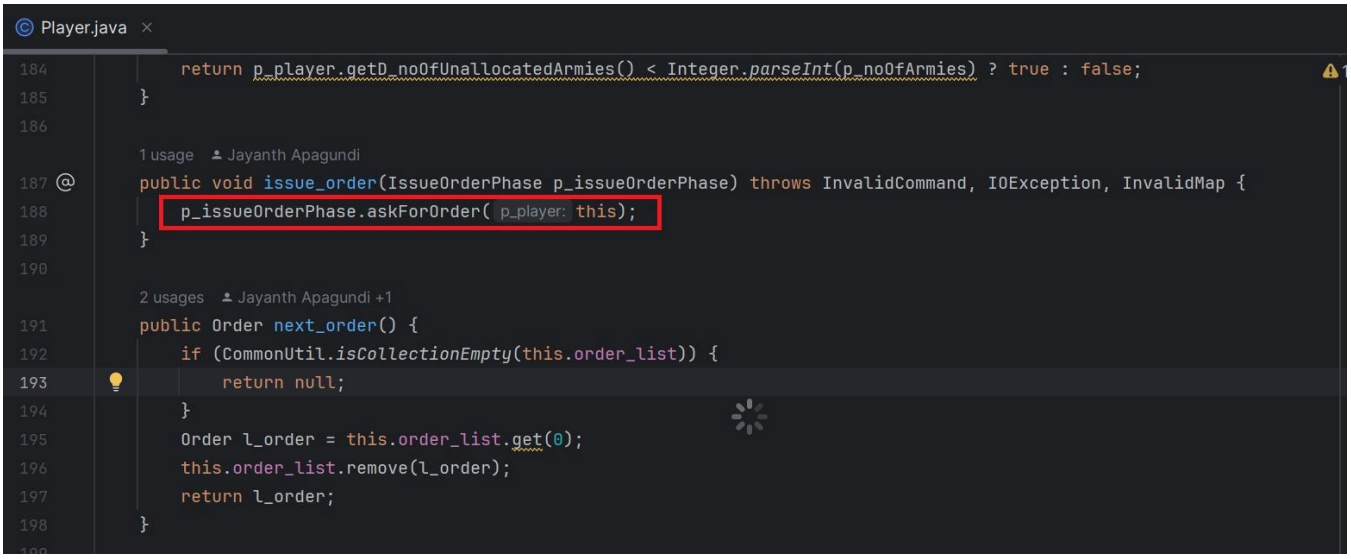
Before

Player.java

```
189     /**
190      * Issues an order for the player
191      * @throws IOException If an I/O error occurs.
192      */
         1 usage
193     public void issue_order() throws IOException {
194         BufferedReader l_reader = new BufferedReader(new InputStreamReader(System.in));
195         PlayerService l_playerService = new PlayerService();
196         System.out.println("\nPlease enter command to deploy reinforcement armies on the map for player : "
197                 + this.getPlayerName());
198         String l_commandEntered = l_reader.readLine();
199         Command l_command = new Command(l_commandEntered);
200
201         if (l_command.getRootCommand().equalsIgnoreCase( anotherString: "deploy") && l_commandEntered.split( regex: " ").lengt
202             l_playerService.createDeployOrder(l_commandEntered, p_player: this);
203         } else {
204             System.out.println(ApplicationConstants.INVALID_COMMAND_ERROR_DEPLOY_ORDER);;
205         }
206     }
207
208     /**
209      * Returns the next order to be executed by the player.
210      * @return The next order, or null if no orders are pending.
211      */
212
         2 usages
```

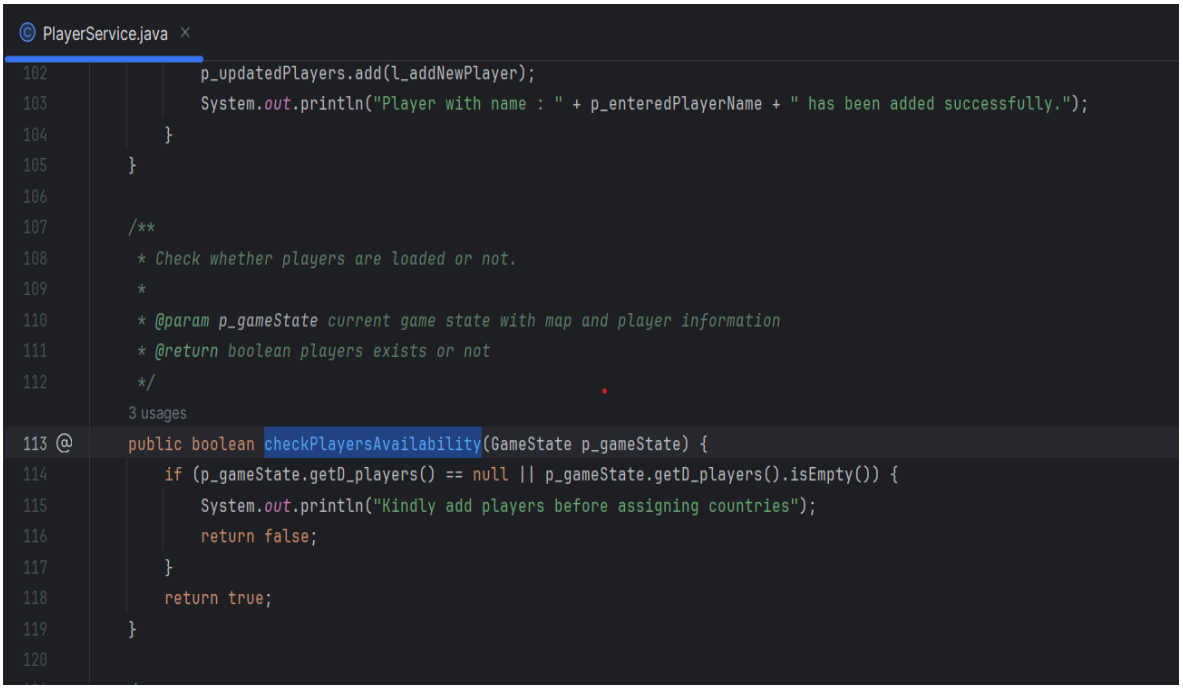
After: In the "after" version, the code utilizes the Command pattern to encapsulate orders as objects. The Player class's issueOrder method now takes an Order object as a parameter and adds it to the player's list of orders. This decouples the player logic from the specifics of order execution, promoting better separation of concerns.

After



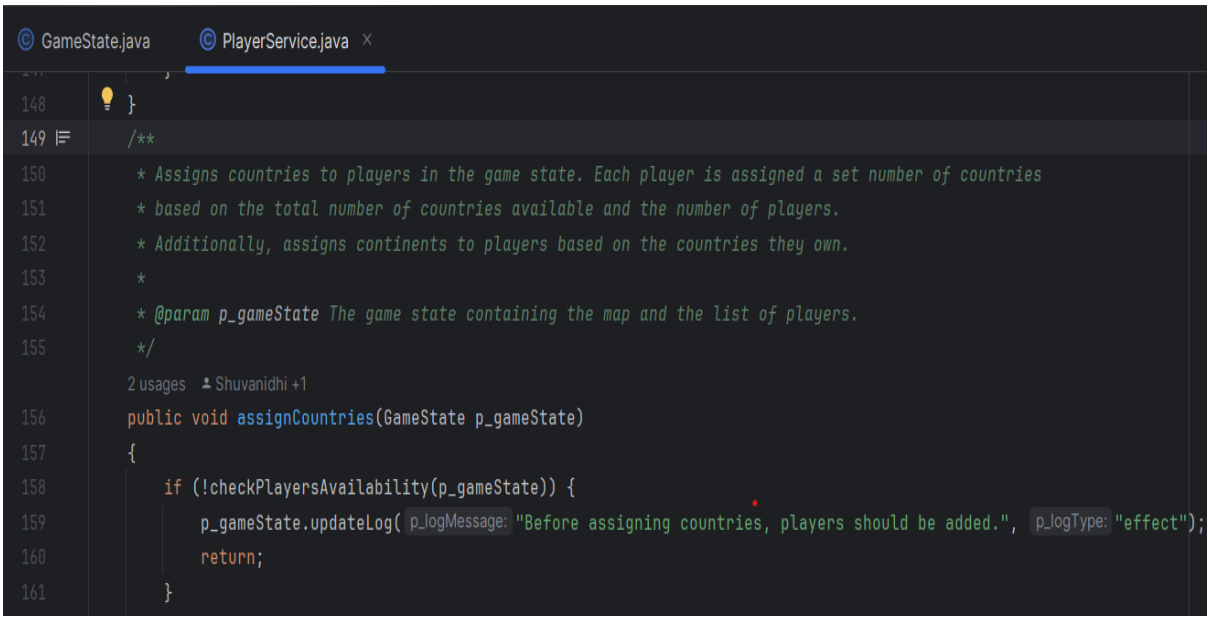
4.Identify and decouple dependencies:

Before: In the "checkPlayersAvailability" method, the before code displayed a message to the console when players were not available.



```
PlayerService.java
102                 p_updatedPlayers.add(l_addNewPlayer);
103                 System.out.println("Player with name : " + p_enteredPlayerName + " has been added successfully.");
104             }
105         }
106
107         /**
108          * Check whether players are loaded or not.
109          *
110          * @param p_gameState current game state with map and player information
111          * @return boolean players exists or not
112          */
            3 usages
113 @       public boolean checkPlayersAvailability(GameState p_gameState) {
114             if (p_gameState.getD_players() == null || p_gameState.getD_players().isEmpty()) {
115                 System.out.println("Kindly add players before assigning countries");
116                 return false;
117             }
118             return true;
119         }
120
```

After: In the after code, this console output was removed to decouple the method from the specific output mechanism.



```
GameState.java   PlayerService.java
148         }
149         /**
150          * Assigns countries to players in the game state. Each player is assigned a set number of countries
151          * based on the total number of countries available and the number of players.
152          * Additionally, assigns continents to players based on the countries they own.
153          *
154          * @param p_gameState The game state containing the map and the list of players.
155          */
            2 usages   Shuvanidhi +1
156         public void assignCountries(GameState p_gameState)
157         {
158             if (!checkPlayersAvailability(p_gameState)) {
159                 p_gameState.updateLog( p_logMessage: "Before assigning countries, players should be added.",  p_logType: "effect");
160                 return;
161             }
```

5. Enhancing Exception Handling:

Before: In the "before" version, the deleteCountry and deleteCountryNeighbours methods did not have any exception handling.

Before

Continent.java

```
134      */
         1 usage
135      public void deleteCountry(Country p_country){
136          if(d_countries==null){
137              System.out.println("No such Country Exists");
138          }else {
139              d_countries.remove(p_country);
140          }
141      }
142
143      /**
144       * Deletes the neighboring relationship of a country with the specified ID within the continent.
145       * @param p_countryId The ID of the country whose neighbors are to be deleted.
146       */
         2 usages
147      public void deleteCountryNeighbours(Integer p_countryId){
148          if (null!=d_countries && !d_countries.isEmpty()) {
149              for (Country c: d_countries){
150                  if (!CommonUtil.isNull(c.d_adjacentCountryIds)) {
151                      if (c.getD_adjacentCountryIds().contains(p_countryId)){
152                          c.deleteNeighbour(p_countryId);
153                      }
154                  }
155              }
156          }
```

After: In the "after" version, both methods now declare that they throw an InvalidMap exception. This indicates that these methods may encounter issues related to map validation and provides a way to handle such exceptions elsewhere in the code.

After

Continent.java

```
134      * @param p_country The country to delete.
135      */
         1 usage  Shuvanidhi +1
136      public void deleteCountry(Country p_country) throws InvalidMap{
137          if(d_countries==null){
138              System.out.println("No such Country Exists");
139          }else {
140              d_countries.remove(p_country);
141          }
142      }
143
144      /**
145       * Deletes the neighboring relationship of a country with the specified ID within the continent.
146       * @param p_countryId The ID of the country whose neighbors are to be deleted.
147       */
         2 usages  Shuvanidhi +1
148      public void deleteCountryNeighbours(Integer p_countryId) throws InvalidMap{
149          if (null!=d_countries && !d_countries.isEmpty()) {
150              for (Country c: d_countries){
151                  if (!CommonUtil.isNull(c.d_adjacentCountryIds)) {
152                      if (c.getD_adjacentCountryIds().contains(p_countryId)){
153                          c.deleteNeighbour(p_countryId);
154                      }
155                  }
156              }
```

″